

Single-Zone IIoT Decentralized Identity Management Framework

Course: Blockchain & Cryptography — Assignment 1 **Team:** Muhammad Shahzaib (23i-2105), Lutfan Shahzad (23i-2114) **Date:** September 2026

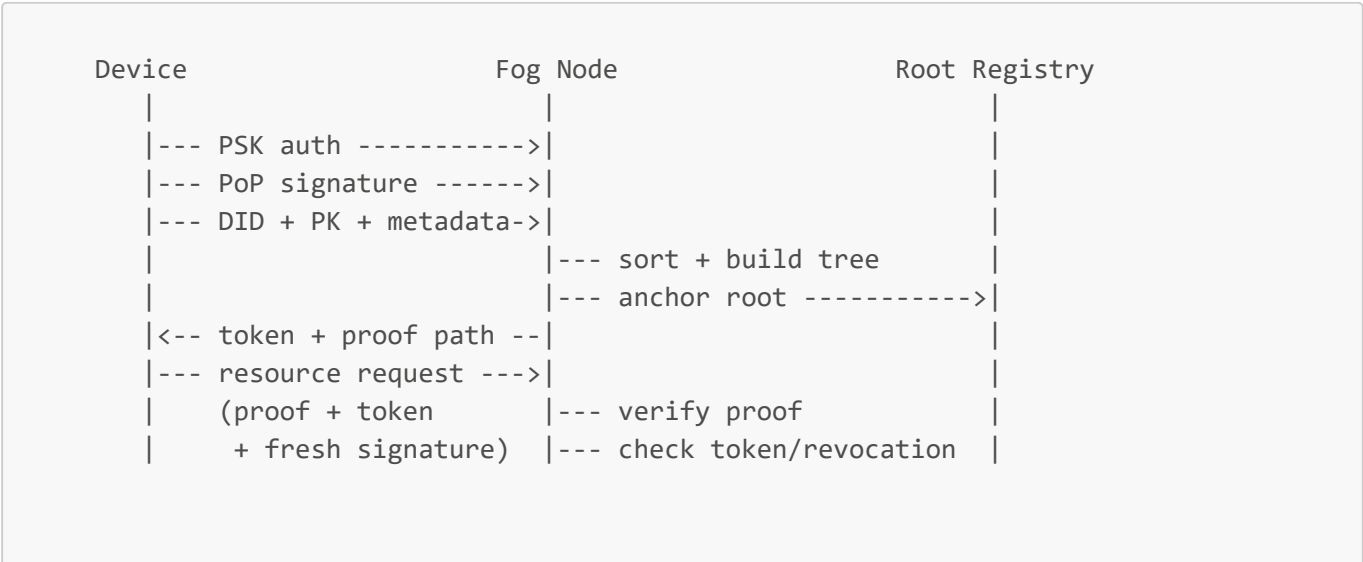
1. Introduction and Architecture

We implement a single-zone IIoT identity-management framework consisting of one fog node and multiple simulated devices. Devices perform only lightweight operations like key generation, hashing, and signing while the fog handles all computationally intensive work: batch aggregation, Merkle tree construction, proof generation, token issuance, verification, and revocation.

1.1 Components

Component	Role
IIoT device (simulated)	Generates DID + ECC keypair, signs PoP challenges, requests tokens, submits proofs
Fog node (Flask)	Authenticates devices, forms batches, builds trees, anchors roots, issues tokens, verifies requests, enforces policy, manages revocation
Root registry (hash chain)	Append-only ledger simulating blockchain anchoring of epoch roots
Policy module	Role → resource → allowed-operations map
Attack suite	Four scripts demonstrating replay, tampering, theft, and revocation
Benchmark + plotter	Measures scalability; produces three required graphs

1.2 Data flow



```
|
|--- apply policy
|<-- ALLOW / DENY -----|
```

1.3 Trust model

- Devices are untrusted until they complete PSK bootstrap + proof-of-possession.
- Fog node is trusted for this assignment (single zone); multi-fog trust is deferred to Assignment 2.
- Root registry is append-only — anchored roots cannot be retroactively altered.
- Private keys never leave devices; the fog only sees public keys and signatures.

1.4 Stack and cryptographic primitives

Layer	Technology
Language	Python 3.14
Fog server	Flask 2.3.3
Transport	HTTPS via self-signed certificate (TLS 1.3)
Crypto library	<code>cryptography</code> (ECDSA P-256, RSA, X.509)
Hashing	<code>hashlib</code> (SHA-256)
State	In-memory dicts + append-only hash chain
Graphs	<code>matplotlib</code>

Primitive	Where used
ECDSA P-256	Device identity keys, fog signing key
SHA-256	Leaf $Ld = H(DID PK)$, internal Merkle nodes, anchor hash
Canonical serialization	$DID " " PK$ before hashing
Fresh nonces	PoP challenge, resource request
Nonce blacklist	<code>USED_NONCES</code> , <code>USED_REQUEST_NONCES</code>
Fog-signed tokens	Temporary tokens (Phase 2)
Hash-chained registry	Epoch anchor records

2. Implementation

2.1 Phase 1 — Registration and Batch Formation

Onboarding sequence. Each device: (1) POSTs `/register/start` with a pre-shared key, receiving a session ID; (2) generates a `did:iot:<name>` and an ECDSA P-256 keypair locally (private key never transmitted); (3) POSTs `/register/challenge` with the public key and receives a fresh 32-byte nonce; (4) signs the nonce and POSTs `/register/complete` with DID, PK, signature, and metadata. The fog verifies: session validity, nonce TTL (60 s), nonce match and non-reuse, public-key match, and ECDSA signature. On success, the device's leaf $Ld = SHA256(DID || " | " || PK_hex)$ is appended to the current batch.

Why PoP matters. Copying a DID and public key is trivial. PoP forces the requester to sign a fresh, fog-generated nonce with the private key matching the claimed public key blocking both impersonation and replay of an old valid signature.

Batch finalization. At the end of a registration window, the fog: collects all leaves, sorts them deterministically (bytewise), builds the Merkle tree once, computes a single root, and anchors it in an append-only hash chain ($\text{anchor_hash} = \text{SHA256}(\text{epoch} || \text{root} || \text{timestamp} || \text{prev_hash})$). It then generates and signs an inclusion proof per device ($\text{Sign_Fog}(\text{leaf} || \text{root} || \text{epoch})$), increments the epoch, and clears the batch. Each device stores its proof package locally and reuses it indefinitely — the epoch root is frozen.

2.2 Phase 2 — Temporary Token Bridging

A device registering mid-epoch has no Merkle proof yet. To bridge this window, the fog issues a short-lived, fog-signed token bound to the device:

```
Issue(d) = DID || PK || scope || issued_at || expires_at || token_id
τd = Sign_Fog( Issue(d) )
```

Demo TTL is 300 s. On every resource request the fog rejects unless: token exists, token signature valid, $\text{now} < \text{expires_at}$, $\text{token_id} \notin \text{REVOCATIONS}$, and token.did/pk match the request. A stolen token is still insufficient, each request includes a fresh device-generated nonce and a signature over $\text{nonce} || \text{resource} || \text{operation}$ using the device's private key. The fog verifies this signature and rejects reused nonces. This converts the token from a bearer credential into a proof-of-possession credential. Once the epoch closes, the device transitions to its permanent Merkle proof.

2.3 Phase 3 — Verification and Resource Authorization

Verification asks *"was this DID + PK registered in a trusted epoch?"*; authorization asks *"may this device perform this operation now?"* A device can pass the first and fail the second.

Pipeline. For each request the fog: (1) resolves $\text{ANCHORED_ROOTS}[\text{epoch}]$; (2) recomputes $\text{Ld} = \text{SHA256}(\text{DID} || \text{PK})$; (3) verifies the Merkle proof against the anchored root; (4) validates the token; (5) checks nonce non-reuse; (6) verifies the request signature $\text{Sign_SKd}(\text{nonce} || \text{resource} || \text{operation})$; (7) applies the policy. Every check must pass for **ALLOW**.

Policy is a two-level map $\text{role} \rightarrow \text{resource} \rightarrow [\text{operations}]$:

```
POLICY = {
  "temperature-sensor": {"temperature": ["WRITE", "READ"]},
  "pressure-sensor":    {"pressure":    ["WRITE", "READ"]},
  "valve-controller":   {"valve":       ["WRITE", "READ"],
                        "production-line": ["STOP"]},
  "camera":             {"stream":      ["READ"]},
}
```

A camera attempting `WRITE stream` is rejected even though its identity is valid i-e the identity $\neq$ authorization demonstration.

2.4 Revocation

Two-layer model:

Layer	Latency	Scope
Local block	Immediate	<code>REVOCATIONS</code> set on fog; token rejected on next request
Epoch exclusion	Next epoch	Device skipped from future batch and root

The old anchored root is never modified while historic proofs remain mathematically verifiable. Access is controlled by current revocation state, not by rewriting history. Both layers are needed: immediate blocking must not wait for the next epoch boundary, and old proofs will always verify against their historic root.

3. Security Analysis

Four attacks implemented in `attacks/attacks.py`:

Attack	Setup	Detected by	Result
Replay of exact request	Capture and resend a valid request verbatim	Nonce blacklist (<code>USED_REQUEST_NONCES</code>)	DENY — nonce reused
Tampered Merkle proof	Flip one bit in a sibling hash	Merkle verifier (recomputed root $\neq$ anchored root)	DENY — proof invalid
Stolen valid token	Attacker holds token + proof but not private key	Request PoP signature check	DENY — request signature invalid
Revoked token reuse	Use an unexpired but revoked token	<code>REVOCATIONS</code> check	DENY — token revoked

Each denial occurs at a distinct, explainable stage in the verification pipeline. No single check is redundant with another.

4. Performance Evaluation

4.1 Methodology

Device counts 5, 10, 25, 50, 100; 5 runs each (25 measurements); fog state reset between runs; traffic over TLS; timings captured client-side with `time.perf_counter()`.

4.2 Results (avg over 5 runs)

Metric	n=5	n=10	n=25	n=50	n=100
Registration (ms)	159	243	234	223	211

Metric	n=5	n=10	n=25	n=50	n=100
Batch processing (ms)	53	79	85	78	84
Verification (ms)	57	81	75	72	73

n=5 is a warm-up outlier after `/reset`; values from n=10 onward are used for trend analysis.

4.3 Interpretation

- **Registration:** 211–243 ms (mean $\approx$ 228). No meaningful growth with N.
- **Batch processing:** 78–85 ms (mean $\approx$ 82). Roughly constant from N=10 to 100. Merkle work is $O(n \log n)$ but is dwarfed by fixed per-request overhead at these sizes.
- **Verification:** 72–81 ms (mean $\approx$ 75). Stable across N. Proof path length is $O(\log n)$, matching the paper's claim.
- **TLS overhead dominates.** Plain-HTTP runs showed reg $\approx$ 20 ms and verify $\approx$ 6 ms — a $\sim 10\times$ difference caused by per-request TLS handshakes (each `requests.post()` opens a fresh session). Production with pooling and TLS session resumption would reduce this by 5–10 $\times$, leaving the identity framework itself lightweight.

At $N \leq 100$, batch-time variation is dominated by TLS handshake jitter rather than tree work. Section 5's HTTP-only comparison isolates and exposes the pure Merkle cost.

5. Batch vs Individual Registration

If each arriving device triggers a tree rebuild, we get **N root updates per batch**, every arrival can invalidate other devices' sibling hashes, and up to **$N(N-1)/2$ stale proofs** require redistribution.

Measured (`perf/batch_vs_individual.py`):

N	Individual (ms)	Batch (ms)	Speedup	Root updates	Stale proofs
5	0.0	0.0	1.6 $\times$	5	10
10	0.2	0.1	3.3 $\times$	10	45
25	1.0	0.1	9.4 $\times$	25	300
50	2.1	0.2	11.6 $\times$	50	1,225
100	5.7	0.3	19.6 $\times$	100	4,950
200	24.4	0.7	34.3 $\times$	200	19,900

Batch registration delivers sub-linear tree-construction cost, one root per epoch instead of N, and zero proof churn within a batch. The saving grows with N which is exactly what a scalable IIoT identity system requires.

6. Problems Encountered and Design Decisions

6.1 Problems

Problem	Resolution
Self-signed TLS caused <code>CERTIFICATE_VERIFY_FAILED</code> in clients	<code>Session.request</code> monkey-patch to disable cert verification (demo only)
Flask CLI (<code>flask run</code>) ignored <code>ssl_context</code>	Switched to <code>python -m fog.server</code>
Benchmark accumulated fog state → wrong trends	Added <code>/reset</code> endpoint called before each run
Tampered-proof attack crashed on single-leaf tree (empty proof)	Register 4 filler devices so victim has a real proof path
Odd leaf count in Merkle tree	Duplicate last leaf (standard convention)

6.2 Design decisions

Decision	Rationale
ECDSA P-256, not RSA	Smaller keys, faster, standard on IIoT hardware
Regular Merkle tree, not SMT	Sufficient for single-zone membership; SMT deferred to Assignment 2
Fresh nonces in every request	Replay protection at every step, not just initial auth
Two-layer revocation	Immediate local block + epoch exclusion: fast response plus long-term consistency
Hash-chain root registry	Demonstrates anchoring immutability without full blockchain overhead
Self-signed TLS	Satisfies the protected-connection requirement in a demo setting

7. Contribution Table

Student	Main components	Important files	Testing performed
Lutfan Shahzad (23i-2114)	Crypto primitives, Merkle tree, fog server core (register, batch, verify, revoke), attacks, performance benchmark	<code>fog/crypto.py</code> , <code>fog/merkle.py</code> , <code>fog/models.py</code> , <code>fog/server.py</code> , <code>attacks/attacks.py</code> , <code>perf/benchmark.py</code>	Unit tests for crypto & Merkle; attack suite; scalability runs; batch-vs-individual comparison
Muhammad Shahzaib (23i-2105)	Device simulator, token module, access policy, TLS transport, README, plots, screenshots	<code>device/simulator.py</code> , <code>fog/policy.py</code> , <code>fog/token.py</code> , <code>fog/gen_cert.py</code> , <code>perf/plot.py</code> , <code>README.md</code>	Integration test of simulator; TLS handshake; policy decisions; graph generation; demo capture

All code is available on GitHub with per-author commit history.

Appendix A — Screenshots

A.1 Fog node startup (HTTPS)

```
Windows PowerShell
PS D:\Semester 7\Blockchain & Cryptography\Assignment_1\iiot_idm> python -m fog.server
* Serving Flask app 'server'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on https://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 847-319-431
```

Fog runs on `https://127.0.0.1:5000` with a self-signed TLS certificate.

A.2 Device registration

```
PS D:\Semester 7\Blockchain & Cryptography\Assignment_1\iiot_idm> python -m device.simulator -n 5
[simulator] starting 5 devices

registered did:iiot:sim-0000    role=temperature-sensor    ok=True
registered did:iiot:sim-0001    role=pressure-sensor      ok=True
registered did:iiot:sim-0002    role=valve-controller     ok=True
registered did:iiot:sim-0003    role=camera               ok=True
registered did:iiot:sim-0004    role=temperature-sensor   ok=True

[simulator] registration done in 614.8 ms

[simulator] temporary tokens issued

[simulator] epoch 1 finalized  root=6faa98ecd49d42d3...

did:iiot:sim-0000    -> ALLOW
did:iiot:sim-0001    -> ALLOW
did:iiot:sim-0002    -> ALLOW
did:iiot:sim-0003    -> DENY
did:iiot:sim-0004    -> ALLOW
PS D:\Semester 7\Blockchain & Cryptography\Assignment_1\iiot_idm>
```

Five simulated devices with mixed roles register sequentially via PSK + PoP.

A.3 Batch finalization and proof verification

```
PS D:\Semester 7\Blockchain & Cryptography\Assignment_1\iiot_idm> python -m device.test_batch
Finalize: {'anchor_hash': 'fae1762fb8bb890f8d55e0fc987821cfc37f57275570bded8a324f18565f9368', 'devices': 4, 'epoch_finalized': 1, 'ok': True, 'root_hex': '951dbe058811dcb92102248249412f8e31b6f3295e5ba928df7b98044dd3fab3'}
Local proof verify for dev-001: True
Proof path length: 2
PS D:\Semester 7\Blockchain & Cryptography\Assignment_1\iiot_idm>
```

Batch finalized; one root anchored; proof path of length 2 verified locally against the anchored root.

A.4 Verification: ALLOW and DENY

```
PS D:\Semester 7\Blockchain & Cryptography\Assignment_1\iiot_idm> python -m device.test_verify
1) WRITE temperature -> {'decision': 'ALLOW', 'ok': True}
2) STOP production-line -> {'decision': 'DENY', 'ok': False, 'reason': 'policy: temperature-sensor cannot STOP production-line'}
PS D:\Semester 7\Blockchain & Cryptography\Assignment_1\iiot_idm>
```

`WRITE temperature` → ALLOW; `STOP production-line` → DENY. Identity ≠ authorization.

A.5 Revocation

```
PS D:\Semester 7\Blockchain & Cryptography\Assignment_1\iiot_idm> python -m device.test_revoke
1) before revoke -> {'decision': 'ALLOW', 'ok': True}
2) revoke -> {'ok': True, 'revoked_device': 'did:iiot:temp-888', 'revoked_devices': ['did:iiot:temp-888'], 'revoked_token': 'a7a2431ced7b4f73b671c225e2de2071'}
3) after revoke -> {'decision': 'DENY', 'ok': False, 'reason': 'token revoked'}
4) status -> {'revoked_devices': ['did:iiot:temp-888'], 'revoked_tokens': ['a7a2431ced7b4f73b671c225e2de2071']}
PS D:\Semester 7\Blockchain & Cryptography\Assignment_1\iiot_idm>
```

Before revoke: ALLOW. After /revoke: DENY — token revoked. Historic proofs remain valid.

A.6 Security attacks

```
PS D:\Semester 7\Blockchain & Cryptography\Assignment_1\iiot_idm> python -m attacks.attacks
[setup] victim proof path length = 3

=== ATTACK 1: Replay exact request ===
1st time : {'decision': 'ALLOW', 'ok': True}
Replayed : {'decision': 'DENY', 'ok': False, 'reason': 'nonce reused'}
Detected : fog nonce validator (USED_REQUEST_NONCES)

=== ATTACK 2: Tampered Merkle proof ===
Result : {'decision': 'DENY', 'ok': False, 'reason': 'proof invalid'}
Detected : Merkle proof verification (root mismatch)

=== ATTACK 3: Stolen valid token ===
Result : {'decision': 'DENY', 'ok': False, 'reason': 'request signature invalid'}
Detected : request proof-of-possession (signature over nonce)

=== ATTACK 4: Revoked token reuse ===
Result : {'decision': 'DENY', 'ok': False, 'reason': 'token revoked'}
Detected : token revocation check
PS D:\Semester 7\Blockchain & Cryptography\Assignment_1\iiot_idm>
```

All four attacks blocked: replay, tampered proof, stolen token, revoked token.

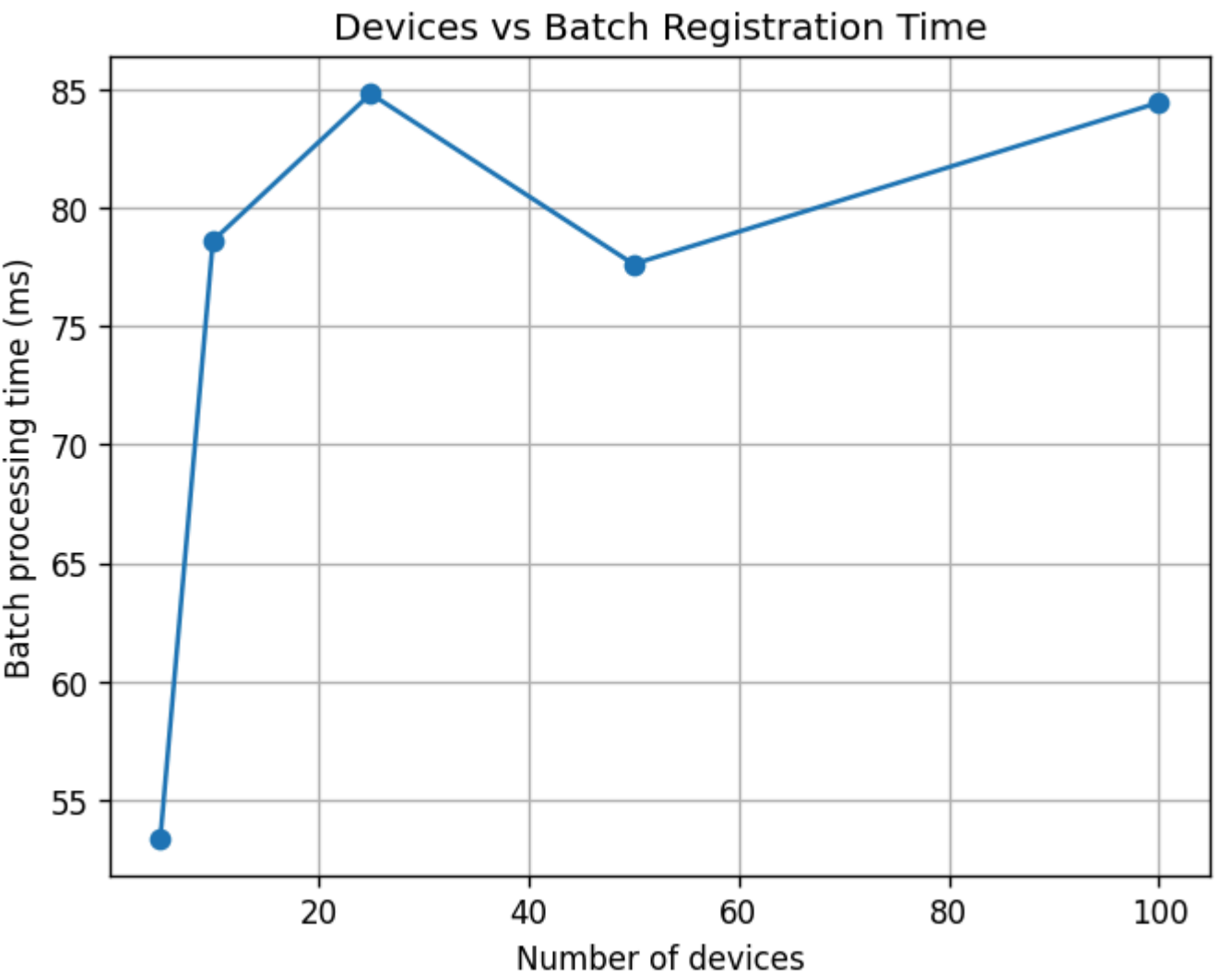
A.7 Performance benchmark

```
PS D:\Semester 7\Blockchain & Cryptography\Assignment_1\iiot_idm> python -m perf.benchmark
[n=5 run=1/5] ... reg=115.7ms batch=36.1ms verify=32.4ms
[n=5 run=2/5] ... reg=125.3ms batch=49.1ms verify=36.6ms
[n=5 run=3/5] ... reg=118.1ms batch=43.1ms verify=58.8ms
[n=5 run=4/5] ... reg=199.0ms batch=64.0ms verify=88.1ms
[n=5 run=5/5] ... reg=235.1ms batch=74.8ms verify=71.3ms
[n=10 run=1/5] ... reg=252.8ms batch=79.2ms verify=86.0ms
[n=10 run=2/5] ... reg=245.3ms batch=83.8ms verify=81.4ms
[n=10 run=3/5] ... reg=242.8ms batch=77.0ms verify=76.7ms
[n=10 run=4/5] ... reg=236.5ms batch=78.0ms verify=85.7ms
[n=10 run=5/5] ... reg=234.9ms batch=75.0ms verify=75.1ms
[n=25 run=1/5] ... reg=235.9ms batch=98.3ms verify=80.0ms
[n=25 run=2/5] ... reg=233.4ms batch=87.1ms verify=71.7ms
[n=25 run=3/5] ... reg=236.0ms batch=79.9ms verify=72.0ms
[n=25 run=4/5] ... reg=234.2ms batch=78.9ms verify=74.3ms
[n=25 run=5/5] ... reg=229.7ms batch=79.8ms verify=75.7ms
[n=50 run=1/5] ... reg=228.7ms batch=82.5ms verify=80.5ms
[n=50 run=2/5] ... reg=217.4ms batch=82.8ms verify=72.4ms
[n=50 run=3/5] ... reg=219.1ms batch=78.9ms verify=77.4ms
[n=50 run=4/5] ... reg=228.4ms batch=94.4ms verify=87.2ms
[n=50 run=5/5] ... reg=221.6ms batch=49.4ms verify=43.6ms
[n=100 run=1/5] ... reg=185.9ms batch=81.7ms verify=71.8ms
[n=100 run=2/5] ... reg=194.4ms batch=88.0ms verify=71.1ms
[n=100 run=3/5] ... reg=227.7ms batch=82.1ms verify=72.4ms
[n=100 run=4/5] ... reg=224.7ms batch=84.4ms verify=72.4ms
[n=100 run=5/5] ... reg=224.2ms batch=85.9ms verify=74.7ms

Saved 25 rows to data/perf_results.csv
PS D:\Semester 7\Blockchain & Cryptography\Assignment_1\iiot_idm>
```

5 runs per device count across 5, 10, 25, 50, 100 devices.

A.8 Scalability graph



Batch-processing time across N under TLS. Absolute values include handshake overhead; pure Merkle cost is isolated in Section 5.